

Payout Destinations, Payables Modal & Promotion Edit Fix — Design

Status: Approved by user 2026-09-16 (conversational brainstorming; design refined through targeted AskUserQuestion rounds, then five adjustments requested by user and incorporated below).

Relationship to prior work

This is **Phase 2** on top of two already-implemented specs:

- [2026-09-07-manual-cafe-payouts-design.md](#) — introduced `CafePayout/CafePayoutItem`, the outstanding-balance calculation, and the admin/owner payout API surface.
- [2026-09-16-finance-settlement-design.md](#) — rebuilt `/admin/cafe-payouts` into the current single finance page (per-café "Owed now", UPI/bank display with a "not independently verified" tooltip, On Hold toggle, payout history), added `CafePayoutAdjustment` for refund clawback, dropped the `payout_verification_status == "verified"` hard gate, and simplified the owner payouts page to one number + one history list. Confirmed implemented via today's commits ([e95d1b7](#), [61b652f](#), [25e5eb7](#), [8c2a119](#), [d2bf95d](#)).

This spec does **not** re-build that page. It closes five specific gaps a fresh codebase audit found still present after that work:

1. The endpoint feeding the payables list/modal (`list_cafes_with_outstanding` / the breakdown endpoint) drops bank fields entirely — only `upiVpa` is returned. A bank-only café shows "no UPI ID on file" even though it has usable payout info.
2. Bank account numbers are **never decrypted for any API response** anywhere in the codebase (`cafe_service.py:122-124`, explicit comment). That means even inside the actual "record payout" flow, an admin paying a bank-only café by bank transfer cannot see the full account number to type into their banking app. Masked-only display makes that café operationally unpayable, not just harder to read.

3. `CafePayout` never snapshots which destination was actually used. If an owner changes their UPI between accrual and payment, there's no structured record of what the admin actually paid to — only optional free-text notes.
4. Owners have no way to edit payout details after onboarding. A legacy, disconnected "Direct Bank Payouts" settings card (old Razorpay-Route/KYC flow) writes a different subset of the same `OwnerPayoutAccount` row and silently clobbers fields the real manual-payout system depends on.
5. The payables "select a payable" modal is a bespoke `<div className="fixed inset-0...">` (`admin/cafe-payouts/page.tsx:157-293`), not the app's shared `components/ui/Modal.tsx`. No `max-h-[90vh]`/scroll on the outer card, no backdrop-click-to-close, no Escape handler — confirmed still true after the finance-settlement rebuild, which touched the page's content but not its container.

Separately, Priority 3 (promotion editing) is unrelated to the payout work: a confirmed one-line-class bug in `PromotionRepository.update()`.

Part A — Payout destination visibility (closes gaps 1–3)

A.1 Data model changes

`OwnerPayoutAccount` — add one column:

column	type	notes
<code>version</code>	Integer, default 1, not null	Incremented server-side by <code>upsert_payout_details()</code> only when a destination field actually changes (no-op saves don't bump it). Gives a stable identity for "which set of details was live when."

`CafePayout` — add seven columns, all nullable, populated once at creation time from the live `OwnerPayoutAccount`, never updated after:

column	type	notes
<code>destination_type</code>	Enum: <code>upi</code> , <code>bank</code>	which method was actually

column	type	notes
		used for this payout
destination_upi_vpa	String, nullable	full UPI (not sensitive on its own — already shown in full elsewhere in the app today)
destination_bank_account_masked	String, nullable	masked form only, e.g. ****1234 — never the decrypted number
destination_bank_ifsc	String, nullable	
destination_account_holder_name	String, nullable	
destination_payout_account_id	UUID FK → owner_payout_accounts.id, nullable	forensic trail: which account row this came from
destination_payout_account_version	Integer, nullable	forensic trail: which version of that row

Pre-existing **CafePayout** rows (created before this migration) get **NULL** across all seven columns. The UI shows "not recorded (pre-dates this feature)" for those — **no backfill is attempted**, since we cannot know what was actually live at a past payout's creation time and manufacturing a value would be worse than admitting the gap.

New table **OwnerAuditLog** (owner-initiated sensitive actions — distinct from the existing **AdminAuditLog**, which is admin-actor-shaped and used elsewhere; not repurposing it):

column	type	notes
id	UUID PK	
owner_id	UUID FK → users.id	
action	String	starts with payout_details.updated
entity_type / entity_id	String / UUID	owner_payout_account / the account id
before_summary	Text	masked representation only — see A.4
after_summary	Text	masked representation only — see A.4
created_at	DateTime	

A.2 Reveal-for-payout (closes gap 2)

Masked display stays the default everywhere (payables list, breakdown view, café detail page) — unchanged from today's policy. A **new, explicit, narrow action** inside the actual "record payout" step:

- **New endpoint** **POST** **/api/v1/admin/cafe-payouts/{cafe_id}/reveal-destination** — admin-role only (same bar as every other admin payout endpoint; no new role tier, per the earlier decision that all current **admin** users already see equivalent data elsewhere). Decrypts the bank account number server-side, just for this response. Returns **{upi_vpa, bank_account_number (full), bank_ifsc, account_holder_name, payout_account_id, payout_account_version, updated_at}**.

- Triggered by an explicit admin gesture — a "Show full details to pay" button inside the record-payout panel, not auto-loaded with the rest of the breakdown. This gives a real user action to log against.
- **Writes an `AdminAuditLog` row** (`action="payout_destination.revealed"`, actor, target cafe, timestamp) — logs the *fact* of the reveal, never the decrypted value itself.
- Frontend holds the revealed value in component-local state only, cleared when the modal closes. Not cached in any global store, not persisted to localStorage.
- **Hard requirement, enforced by code review not just convention:** the decrypted bank account number must never appear in application logs, audit logs, analytics events, or error messages. `payout_encryption.py`'s decrypt function's call sites in this new endpoint must not pass through any `logger.info/logger.debug` call with the raw value.

A.3 Staleness detection (closes gap 3's race condition)

Flow:

1. Admin opens a payable → breakdown response includes the masked display fields **and** `payout_account_version` (not yet revealed/decrypted at this point — this is the version number from A.1, safe to expose).
2. If admin clicks "Show full details to pay" → the reveal endpoint (A.2) returns the same `payout_account_version` alongside the decrypted values, confirming what version they're now looking at.
3. On "Record Payout" submit, the request body includes `expected_payout_account_version` — whatever version the admin last saw (from step 1 or 2, whichever is more recent in the UI flow).
4. Backend re-fetches the live `OwnerPayoutAccount` for that owner at submission time and compares versions. **Mismatch** → **reject with 409** and a specific message: "Payout details changed since you opened this payable. Please refresh and confirm the new destination before recording this payout." Frontend surfaces this, forces a re-fetch of the current destination, and requires the admin to explicitly re-view (and, if paying by bank, re-reveal) before resubmitting.
5. On success, `create_payout()` populates all seven `CafePayout` snapshot columns from the now-confirmed-current `OwnerPayoutAccount`.

A.4 Owner-side payout details editor (closes gap 4)

Retire the legacy KYC path — conditionally, not unconditionally. Before any deletion, the implementation plan must run a repo-wide dependency audit grepping for: `owner_payouts`, `/setup`, `OwnerPayoutService`, `KYC`, `OwnerPayoutAccount`, `payout_details`, and inspect every hit — frontend, backend, scheduled jobs, migrations, webhooks, and any external/API consumer. **Removal happens in the same change only if that audit proves zero other consumers.** If anything still depends on it, the endpoint is deprecated (returns 410, or is feature-flagged off) instead of deleted, and a follow-up ticket is filed — never a silent "probably fine" deletion of financially-adjacent code.

Assuming the audit clears it:

- Delete `PayoutSetupCard.tsx` and its usage in `owner/settings/page.tsx`.
- Delete the backend `/setup` route, `OwnerPayoutService`, and `PayoutAccountCreateRequest` schema.

New real editor, backed by the same `upsert_payout_details()` path onboarding already uses (which already correctly resets verification status on change — kept):

- **New endpoint** `GET /api/v1/owner/payout-details` — current details (UPI in full, bank masked), last-updated timestamp, current `version`.
- **New endpoint** `PATCH /api/v1/owner/payout-details` — body: new UPI and/or bank fields + **current account password**. Backend verifies the password first (same password-hash check used elsewhere in auth); wrong password → 401, **no change made, no email sent, no audit row written**. On success: calls `upsert_payout_details()` (bumping `version` per A.1), fires a "your payout details changed" confirmation email via the existing Resend/SES-backed `NotificationService` (informational, sent after the fact — not a blocking pre-confirmation step), and writes an `OwnerAuditLog` row.
- **New frontend component** `PayoutDetailsCard.tsx` in `owner/settings/page.tsx` (replacing the deleted legacy card): shows current UPI/masked-bank + "last updated," an Edit button opening a form (destination fields + password field), submits to the PATCH endpoint, surfaces the real server error on failure, refetches on success.

Audit log content — hard requirement: `before_summary/after_summary` on `OwnerAuditLog`, and any before/after content in `AdminAuditLog` entries touching

payout data, contain **only** masked representations, e.g. UPI: oldupi@upi / Bank: ****1234 / IFSC HDFC0001234. Never a decrypted account number, never the password used to authorize the change, never any other authentication secret.

A.5 Payables endpoints (closes gap 1)

- `list_cafes_with_outstanding` (feeds the payables list) — extended to return `hasUpi: bool` / `hasBank: bool` flags (not full bank details at list-scope, to keep that response lightweight) so the list can correctly show "has payout info on file" for bank-only cafés instead of a false "no UPI ID on file" warning.
- The breakdown endpoint (feeds the opened payable / modal) — extended to return the masked UPI + masked bank fields + `payout_account_version`, per A.3.

Part B — Payables modal rebuild (closes gap 5)

Rebuilt on top of the existing `components/ui/Modal.tsx`, which already correctly implements `max-h-[90vh]` + internal scroll, Escape-to-close, backdrop-click-to-close, focus trap, body-scroll lock, and `role="dialog"/aria-modal` — none of which the current bespoke `<div>` has. This is a container swap, not new layout math.

Content reorganized (preserving every existing field/action from the current implementation — outstanding amount, booking/adjustment breakdown, UTR field, payment method select, date, required proof-image upload, admin note, notes, hold toggle):

1. **Header** — café name + status + close button.
2. **Financial summary** — outstanding amount, breakdown of contributing bookings/adjustments (kept scrollable within its own sub-region as today).
3. **Payout destination** — masked UPI/bank by default, "Show full details to pay" reveal button (A.2), "last updated" timestamp, staleness re-check (A.3) surfaced inline if it fires.
4. **Record payout form** — existing fields (UTR, payment method, date, proof image, notes) **plus** a required destination-method selection (UPI vs bank, feeding `destination_type`) **plus** a new required checkbox: *"I confirm this payment was made externally using the destination shown above."* — distinct from the UTR

field, separating "admin clicked pay" from "admin actually transferred money and is recording it."

5. **Payout history** — existing history list surfaced here rather than requiring navigation away.

Part C — Promotion edit fix (unrelated to payouts)

Root cause (confirmed via static trace, `promotion_repository.py:78-87`):

```
for field, value in update_data.items():
    if hasattr(promo, field) and value is not None:
        setattr(promo, field, value)
```

`update_data` already comes from `update_in.model_dump(exclude_unset=True)` — so every key present represents a field the frontend explicitly sent, including an explicit `None` meaning "clear this field" (e.g. removing "Max Redemptions" sends `maxUses: null`; clearing a KHELO code sends `kheLoCode: null`). The `and value is not None` guard silently drops exactly these clearing intents — the save appears to succeed, but the old value reappears on refetch.

Fix: remove the `value is not None` condition. Since `exclude_unset=True` already filters to explicitly-sent fields, presence in `update_data` is itself sufficient justification to apply the value, `None` included:

```
for field, value in update_data.items():
    if hasattr(promo, field):
        setattr(promo, field, value)
```

Authorization (`cafe.owner_id == owner_id` check in `PromotionService.update_promotion`) is already correctly implemented server-side and does not trust a client-supplied cafe ID — **no change needed there.**

Edge cases

Already handled by existing (Phase 1) infrastructure — verified, not re-solved here: no-destination-at-all blocks payout; concurrent double-payout prevented by the `CafePayoutItem.platform_fee_id` unique constraint; payout-on-hold blocks payout; refund-after-payout nets into a future payout via `CafePayoutAdjustment` without mutating past records.

New edge cases this design introduces:

- Owner changes details after a payable accrues but before actual payout → the eventual payout snapshots whatever is live **at payout time**, which is correct and matches the brief's own worked example.
- Admin views a payable, owner changes destination, admin submits → 409 staleness rejection (A.3), forces re-confirmation. This is the concrete fix for the "paid the old destination without realizing" risk.
- Wrong password on the owner payout-details edit form → 401, no partial write, no email, no audit row.
- Multiple cafés under one owner share one `OwnerPayoutAccount` (existing, deliberate design — confirmed, not a bug) — each café's payables modal will correctly show the identical destination; UI adds a small "shared across your other cafés" note for clarity, not a functional change.
- Legacy KYC endpoint still has live consumers when the dependency audit runs → deprecate (410 / feature flag), do not delete; file a follow-up.

Testing plan

Backend (pytest):

- `create_payout()` correctly populates all seven snapshot columns from the live `OwnerPayoutAccount` at creation time.
- Reveal endpoint: returns decrypted value only for admin role; writes exactly one `AdminAuditLog` row with no decrypted value in its fields; asserts no decrypted value appears in any log call (can be checked by asserting the mock/captured decrypted value string never appears in serialized log output during the test).
- Staleness check: payout submission with a stale `expected_payout_account_version` is rejected 409 and creates no `CafePayout` row; submission with the current version succeeds.
- Owner payout-details PATCH: wrong password → 401, no DB change, no email sent (mock the notification call), no audit row; correct password → 200, `version` bumped exactly when a field actually changed (not on a no-op resubmit of

identical values), confirmation email fired, `OwnerAuditLog` row written with masked-only content.

- `PromotionRepository.update()` regression test: explicitly clearing `maxUses/kheIoCode` to `None` persists across a refetch.
- Payables list/breakdown responses correctly reflect `hasBank`/masked bank fields for a bank-only café (previously showed "no UPI on file").
- Legacy KYC endpoint: either fully removed (if audit clears it) with no remaining route, or explicitly returns 410 if deprecated instead.

Frontend (manual, in-browser verification):

- Modal: Escape closes it, backdrop click closes it, content scrolls internally on a short viewport (both a resized-down desktop window and an actual mobile width), close button always reachable.
- Full record-payout flow for a bank-only café: reveal button surfaces the actual account number, payout records successfully, snapshot fields correctly show on the resulting history entry.
- Owner settings: edit UPI, wrong password rejected with a visible error, correct password succeeds, confirmation email arrives (verify via Resend/SES logs or a test inbox), old legacy card is gone.
- Promotion edit: clear "Max Redemptions," save, reload the page, confirm it stays cleared (not reverted).

Regression: full existing suite (`pytest`, `tsc --noEmit`, `next build`) must stay green throughout.

Migration

One new Alembic migration:

- `owner_payout_accounts`: add `version` (Integer, not null, default 1).
- `cafe_payouts`: add the seven nullable destination-snapshot columns.
- New table `owner_audit_log`.

Purely additive. No backfill of historical `CafePayout` rows (see Part A.1 rationale). Legacy KYC table/columns (if any exist beyond what's already documented as dead in the finance-settlement design) are left untouched regardless of whether the legacy *endpoint* is deleted — schema cleanup of genuinely orphaned columns is explicitly out of scope for this change, to keep the migration's blast radius small and reversible.

Out of scope (explicit non-goals)

- Introducing a distinct "super admin" role/tier — confirmed with user, current flat `admin` role is the authorization bar for all payout-destination visibility in this design.
- Audit-logging admin *views* of masked payout destinations (only writes: detail changes, payout creation, hold toggle, and the new reveal action are logged).
- Any change to Razorpay Route, `PlatformFee.transfer_status`, or re-enabling automated payouts — untouched, per the finance-settlement design's existing non-goals.
- OTP/SMS-based confirmation for payout-detail changes — no SMS infrastructure exists; password re-entry + email notification was the confirmed choice.
- Schema cleanup of any legacy/dead columns beyond what this change's own migration adds.